Question : 1

Write a query to display all Engineering employees ordered by salary from highest to lowest.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT employee_name, salary
FROM Employees
WHERE department = 'Engineering'
ORDER BY salary DESC;
```

Question : 2

Write a query to find the number of employees in each department.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT department, COUNT(*) AS employee_count
FROM Employees
GROUP BY department;
```

Question : 3

Write a query to find the second-highest distinct salary.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT MAX(salary) AS second_highest_salary
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Question : 4

Write a query to find employees earning more than the company average salary.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT employee_name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Question : 5

Write a query to find the highest salary in each department.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT department, MAX(salary) AS highest_salary
FROM Employees
GROUP BY department;
```

Question : 6

Write a query to find employees whose salary is higher than their manager salary.

Employees table

employee_id	employee_name	department	salary	manager_id
101	Aarav	Engineering	95000	105
102	Meera	Engineering	72000	105
103	Kabir	Sales	65000	106
104	Isha	Sales	92000	106
105	Rohan	Engineering	90000	None
106	Neha	Sales	88000	None

QUERY

```
SELECT e.employee_name, e.salary,
       m.employee_name AS manager_name,
       m.salary AS manager_salary
FROM Employees e JOIN Employees m
ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Question : 7

Write a query to display each order with the customer name.

Customers

customer_id	customer_name	city
1	Aditi Sharma	Delhi
2	Rahul Verma	Mumbai
3	Simran Kaur	Chandigarh
4	Arjun Nair	Bengaluru

Orders

order_id	customer_id	amount	status
501	1	2400	Delivered
502	1	3200	Delivered
503	2	1800	Delivered
504	2	4100	Pending
505	3	950	Cancelled

QUERY

```
SELECT o.order_id, c.customer_name, o.amount, o.status
FROM Orders o JOIN Customers c
ON o.customer_id = c.customer_id;
```

Question : 8

Write a query to find customers who have never placed an order.

Customers

customer_id	customer_name	city
1	Aditi Sharma	Delhi
2	Rahul Verma	Mumbai
3	Simran Kaur	Chandigarh
4	Arjun Nair	Bengaluru

Orders

order_id	customer_id	amount	status
501	1	2400	Delivered
502	1	3200	Delivered
503	2	1800	Delivered
504	2	4100	Pending
505	3	950	Cancelled

QUERY

```
SELECT c.customer_id, c.customer_name
FROM Customers c LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Question : 9

Write a query to find customers who placed more than one order.

Customers

customer_id	customer_name	city
1	Aditi Sharma	Delhi
2	Rahul Verma	Mumbai
3	Simran Kaur	Chandigarh
4	Arjun Nair	Bengaluru

Orders

order_id	customer_id	amount	status
501	1	2400	Delivered
502	1	3200	Delivered
503	2	1800	Delivered
504	2	4100	Pending
505	3	950	Cancelled

QUERY

```
SELECT c.customer_name, COUNT(*) AS order_count
FROM Customers c JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(*) > 1;
```

Question : 10

Write a query to find the highest-spending customer based only on delivered orders.

Customers

customer_id	customer_name	city
1	Aditi Sharma	Delhi
2	Rahul Verma	Mumbai
3	Simran Kaur	Chandigarh
4	Arjun Nair	Bengaluru

Orders

order_id	customer_id	amount	status
501	1	2400	Delivered
502	1	3200	Delivered
503	2	1800	Delivered
504	2	4100	Pending
505	3	950	Cancelled

QUERY

```
SELECT c.customer_name, SUM(o.amount) AS total_spending
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.status = 'Delivered'
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spending DESC
LIMIT 1;
```

Question : 11

A table has 100 million records. You want to remove all rows as quickly as possible while keeping the table. Which SQL command will you use?

Interview Answer

- Use TRUNCATE when all rows must be removed quickly.
- It keeps the table structure but removes the data.
- It is generally faster than DELETE for full-table cleanup.

QUERY / EXAMPLE

```
TRUNCATE TABLE Employees;
```

Question : 12

A new intern should only be able to view the Employees table but should not modify it. Which SQL command will you use?

Interview Answer

- Use GRANT to give specific access.
- SELECT permission allows reading data from Employees.
- Do not grant INSERT, UPDATE or DELETE permissions.

QUERY / EXAMPLE

```
GRANT SELECT ON Employees
TO intern_user;
```

Question : 13

An alias created in the SELECT list cannot be referenced in the WHERE clause of the same query. How does SQL's logical execution order explain this?

- WHERE is evaluated before SELECT.
- The alias does not exist yet when WHERE runs.
- Use the expression again, a subquery, or a CTE.

QUERY / EXAMPLE

```
-- Alias is created after WHERE is evaluated

SELECT salary * 12 AS annual_salary
FROM Employees
WHERE annual_salary > 600000; -- invalid in many DBs
```

Question : 14

A ranking query contains duplicate salaries. How will ROW_NUMBER(), RANK(), and DENSE_RANK() assign values differently?

Sample ranking output

Employee	Salary	ROW_NUMBER	RANK	DENSE_RANK
Aman	90000	1	1	1
Neha	80000	2	2	2
Ravi	80000	3	2	2
Simran	70000	4	4	3

QUERY / EXAMPLE

```
SELECT employee, salary,
       ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_no,
       RANK()       OVER(ORDER BY salary DESC) AS rank_no,
       DENSE_RANK() OVER(ORDER BY salary DESC) AS dense_rank_no
FROM Employees;
```

- ROW_NUMBER gives a unique sequence even when salaries tie.
- RANK gives same rank to ties but leaves gaps.
- DENSE_RANK gives same rank to ties without gaps.

Question : 15

A table contains duplicate and NULL email values. How will COUNT(*), COUNT(email), and COUNT(DISTINCT email) differ?

QUERY / EXAMPLE

```
SELECT COUNT(*) AS total_rows,
       COUNT(email) AS non_null_emails,
       COUNT(DISTINCT email) AS
unique_non_null_emails
FROM Customers;
```

- COUNT(*) counts every row.
- COUNT(email) counts only rows where email is not NULL.
- COUNT(DISTINCT email) counts unique non-NULL emails only.

Question : 16

A report contains missing values across primary_phone, alternate_phone, and emergency_phone. How would you return the first available value and show 'Not Available' when all three are NULL?

QUERY / EXAMPLE

```
SELECT customer_name,
       COALESCE(primary_phone, alternate_phone,
                emergency_phone, 'Not Available')
AS contact_phone
FROM Customers;
```

- COALESCE returns the first non-NULL value.
- Useful when multiple optional columns can contain the required value.
- The final text value works as a fallback when all columns are NULL.

Question : 17

Write a query to find each user's previous login date.

User_Logins

user_id	login_date	session_minutes
1	2024-07-01	35
1	2024-07-02	42
1	2024-07-03	28
2	2024-07-01	20
2	2024-07-03	25
3	2024-07-02	45

QUERY

```
SELECT user_id, login_date,
      LAG(login_date) OVER (
          PARTITION BY user_id
          ORDER BY login_date
      ) AS previous_login
FROM User_Logins;
```

Question : 18

A salary report must classify employees into Low, Medium, and High salary bands inside the SELECT output. Which conditional expression would you use, and why is it preferable to IF here?

QUERY / EXAMPLE

```
SELECT employee_name, salary,
       CASE
           WHEN salary >= 80000 THEN 'High'
           WHEN salary >= 50000 THEN 'Medium'
           ELSE 'Low'
       END AS salary_band
FROM Employees;
```

- Use CASE because it works inside SELECT, ORDER BY and aggregate expressions.
- IF is database-specific and usually controls procedural flow.
- CASE is more portable for row-wise conditional output.

Question : 19

A percentage calculation divides achieved_sales by target_sales, but some targets are zero. How would you prevent a divide-by-zero error while preserving those rows?

- NULLIF(target_sales, 0) turns zero into NULL.
- Dividing by NULL returns NULL instead of crashing the query.
- Rows are preserved, so missing or invalid percentage can be handled later.

QUERY / EXAMPLE

```
SELECT employee_id,
       achieved_sales * 100.0 / NULLIF(target_sales, 0) AS achievement_pct
FROM Sales_Targets;
```

Question : 20

Write a query to delete duplicate records while keeping the newest record for each email.

Customer_Records

customer_id	customer_name	email	created_at
301	Nikhil Jain	nikhil@email.com	2024-01-12 10:15
302	Sara Ali	sara@email.com	2024-02-03 09:10
303	Nikhil Jain	nikhil@email.com	2024-05-21 18:30
304	Dev Patel	dev@email.com	2024-06-11 12:45

```
WITH ranked AS (
    SELECT customer_id,
           ROW_NUMBER() OVER (
               PARTITION BY email
               ORDER BY created_at DESC
           ) AS rn
    FROM Customer_Records
)
DELETE FROM Customer_Records
WHERE customer_id IN (
    SELECT customer_id
    FROM ranked
    WHERE rn > 1
);
```